



DATAFLOWX · STAJ PROJESİ BRIEF'İ

PulseGuard

Hafif Filo İzleme & Olay Toplama Sistemi — config-driven bir ajan, tek-yönlü & imzalı veri akışı ve web dashboard ile uçtan uca bir ürün deneyimi.

Go

React + TypeScript

REST API

Docker

Gözlemlenebilirlik

Güvenli Veri Akışı

Süre: ~20 iş günü **Seviye:** Üniversite / temel **Çıktı:** Çalışan sistem + dashboard + demo + mimari doküman

1 · Neden Bu Proje?

Bu staj projesinin amacı sana “bir görev listesi” vermek değil; **gerçek bir ürünün nasıl düşünüldüğünü, tasarlandığını ve parçalara ayrıldığını** kendi ellerinle deneyimletmek. 20 günün sonunda çalışan bir sistemin, onu anlatan bir dokümanın ve “neden böyle yaptım?” sorusuna verebileceğin sağlam cevapların olacak.

Şirket ekosistemi (bağlam)

DataFlowX, birbirinden yalıtılmış ağlar arasında **güvenli veri taşıma** ve **sistem altyapısı izleme** üzerine ürünler geliştiriyor. Senin projen bu ürünlerin hiçbirinin kopyası değil — ama üçünün de *mühendislik refleksinden* ilham alıyor:

Ekosistem ürünü	Ne yapar	PulseGuard’a kattığı fikir
diode	İki izole ağ arasında veriyi yalnızca <i>tek yöne</i> akıtan güvenli veri diyotu.	Ajanın veriyi yalnızca dışarı push etmesi , asla dışarıdan bağlantı kabul etmemesi ve gönderileni imzalaması .
dfx-healthcheck	CPU/RAM/disk/mount izleyen, eşik aşımında alarm üreten ve rapor çıkaran Go ajanı.	Ajanın config-driven check’ler koşması, eşik/alarm mantığı ve raporlama.
system-cli	Ekosistem için ortak, alt-komutlu bir sistem/DevOps yardımcı aracı.	Sistemi yöneten küçük ama gerçek bir komut satırı aracı (<code>pulsectl</code>).

Önemli

Bu ürünlerin kaynak kodunu kopyalaman **beklenmiyor**. Onlar sadece ilham. Senden beklenen; aynı mühendislik sorularını kendi projende sorup kendi cevabını üretmen.

2 · Öğrenim Hedefleri

Bu projeyi bitirdiğinde şunları **yapabiliyor ve anlatabiliyor** olmalısın:

- **Mimari düşünme:** Bir sistemi neden bağımsız parçalara böleriz? İki parça birbiriyle hangi “sözleşme” (arayüz) üzerinden konuşur?
- **Tek-yönlü & güvenli veri akışı:** Veri neden tek yöne akıtılır, bütünlüğü nasıl garanti edilir (imza/hash), araya giren biri veriyi değiştirirse nasıl anlarız?
- **Dayanıklılık (resilience):** Karşı taraf çökerse veri kaybolmamalı. Yerel kuyruk, tekrar deneme (retry), geri-basınç (backpressure) nedir?
- **Config-driven tasarım:** Davranışı koda değil konfigürasyona gömmek neden değerlidir?
- **Modern full-stack:** Go ile servis/CLI, REST API tasarımı, React + TypeScript ile veri gösteren bir arayüz.
- **Ürün mentalitesi:** “Çalışıyor” ile “ürün kalitesinde” arasındaki fark: hata yönetimi, loglama, dokümantasyon, kurulabilirlik.

3 · Ürün Vizyonu & Kullanım Senaryoları

PulseGuard nedir? Bir kurumun onlarca sunucusuna kurulan küçük bir “nöbetçi” ajan, her makinenin sağlığını izler; bir sorun (yüksek CPU, dolan disk, düşen servis) gördüğünde bunu merkezi bir toplayıcıya *güvenle bildirir*; sistem yöneticisi de tek bir web ekranından tüm filoyu izler ve rapor alır.

Örnek senaryolar

- **Nöbetçi:** 09:14’te `web-03` sunucusunda disk %92’ye ulaşır → ajan bir *ALARM* olayı üretir → toplayıcıya iletir → dashboard’da kırmızı yanar, yönetici e-posta/webhook uyarısı alır.
- **Kesintiye dayanıklılık:** Toplayıcı 10 dakika bakımdayken ajan olayları yerelde biriktirir; toplayıcı geri gelince kaldığı yerden gönderir, hiçbir veri kaybolmaz.
- **Denetim/rapor:** Yönetici “son 24 saat, tüm hostlar” için PDF/CSV rapor indirir.
- **Operasyon:** Yönetici sunucuya SSH ile bağlanıp `pulsectl status` ile ajanın durumunu, `pulsectl config validate` ile config’in geçerliliğini kontrol eder.

ARAŞTIRILACAK SORULAR

- “Push” (ajan veriyi gönderir) ile “pull” (merkez veriyi sorar) izleme modelleri arasındaki fark nedir? Bu projede neden **push** daha uygun? (İpucu: diode’un tek-yönlülüğü.)
- Prometheus, Zabbix, Nagios gibi hazır izleme araçları bu işi nasıl yapıyor? Onların ajan mimarisi nasıl çalışıyor?
- Bir “olay (event)” ile bir “metrik (metric)” arasındaki fark nedir? Sen hangisini/ikisini de mi toplamalısın?

4 • Mimari Genel Bakış

Sistem, birbirinden bağımsız geliştirilebilen ve test edilebilen **dört parçadan** oluşur. Her parçanın *tek bir net görevi* vardır ve diğerleriyle sadece tanımlı bir arayüz üzerinden konuşur.



Tek-yönlülük ilkesi (diode ilhamı)

Ajan dışarıdan **hiçbir bağlantı kabul etmez** — dinleyen bir portu yoktur. Sadece kendisi toplayıcıya doğru bağlantı açar. Böylece “merkez ele geçirilse bile ajana/ makineye geri sızamaz”. Bu, güvenlik mimarisinin temel bir dersidir.

ARAŞTIRILACAK SORULAR

- Bir sistemi neden monolitik tek parça yerine ayrı servislere böleriz? Avantaj/ dezavantajları neler?
- Ajan ile toplayıcı arasındaki “sözleşme” (veri formatı) nasıl tanımlanmalı? JSON şeması nasıl tasarlanır, ileride değişirse (versiyonlama) ne olur?
- SQLite ne zaman yeterli, ne zaman PostgreSQL'e geçmek gerekir?

5 · Bileşen Bileşen Gereksinimler

① Agent (Go)

- Bir **YAML config** dosyası okur: hangi check'ler, hangi aralıkla, hangi eşiklerle çalışacak; toplayıcı adresi nedir.
- Periyodik olarak check'leri koşar: en az **CPU, RAM, Disk**; ayrıca *özel komut çalıştırma* veya *HTTP endpoint kontrolü* gibi genişletilebilir bir check tipi.
- Eşik aşımında bir **olay** üretir (seviye: INFO / WARNING / ERROR).
- Olayları **yerel kuyruğa** yazar ve toplayıcıya batch halinde push eder; toplayıcı erişilemezse **tekrar dener**, veri kaybetmez.
- Gönderdiği her batch'i **imzalar / hash'ler** (bütünlük kanıtı).

▢ ARAŞTIRILACAK SORULAR

- Go'da periyodik iş nasıl çalıştırılır (`time.Ticker` , goroutine)? Aynı anda birden çok check koşarsa nelere dikkat etmeli?
- Check'leri "eklenebilir" yapmanın temiz yolu nedir? (İpucu: Go `interface` .) Yeni bir check tipi eklerken mevcut kodu değiştirmek zorunda kalmamalısn — neden?
- Yerel kuyruk için: bellekte tutmak mı, diske yazmak mı? Ajan yeniden başlarsa biriken olaylar ne olmalı?

② Collector + REST API (Go)

- Ajanlardan gelen imzalı olay batch'lerini kabul eder, **bütünlüğü doğrular**, geçersizse reddeder.
- Olayları kalıcı olarak saklar (SQLite).
- Dashboard'ın kullanacağı REST uç noktalarını sunar: host listesi, host detayı, olay/alarm sorgulama (zaman aralığı, seviye filtresi), eşik ayarları.

▢ ARAŞTIRILACAK SORULAR

- İyi bir REST API neye benzer? Kaynak isimlendirme, HTTP metodları, durum kodları (200/201/400/401/409...) ne anlama gelir?
- Aynı batch iki kez gelirse ne olmalı? (İpucu: "idempotency" / çift kayıt engelleme.)
- Ajan toplayıcıya kendini nasıl tanıtır/yetkilendirir? (token? API key?) Güvenli olması için neye dikkat etmeli?

③ Dashboard (React + TypeScript)

- Tüm hostların özet durumunu gösteren bir **filo görünümü** (yeşil/sarı/kırmızı).
- Bir host'a tıklayınca **detay**: son metrikler, olay/alarm geçmişi (zaman çizelgesi/grafik).
- Eşiklerin görüntülenmesi (ve mümkünse düzenlenmesi).
- Seçilen aralık için **PDF veya CSV rapor** indirme.

ARAŞTIRILACAK SORULAR

- TypeScript, düz JavaScript'e göre ne kazandırır? "Tip" hataları neyi engeller?
- Dashboard veriyi ne sıklıkta yenilemeli? Polling nedir, WebSocket ne zaman gerekir?
- Grafik için hangi kütüphane? (ör. Recharts, Chart.js) Seçimini neye göre yaparsın?

④ pulsectl (CLI, Go)

- Alt-komutlar: `pulsectl status` (ajan çalışıyor mu, son gönderim ne zaman), `pulsectl config validate` (config geçerli mi), `pulsectl report` (hızlı özet).
- Anlaşılır çıktı, hatalı kullanımda net hata mesajı ve doğru *exit code*.

ARAŞTIRILACAK SORULAR

- Go'da CLI yazmak için `flag` paketi mi, `cobra` gibi bir kütüphane mi? Artı/eksileri?
- İyi bir CLI'nin özellikleri neler? (exit code, `--help`, insan-okur / makine-okur çıktı...)

6 · Fonksiyonel Gereksinimler

Aşağıdakiler **Zorunlu (çekirdek)** ve **Opsiyonel (ileri hedef)** olarak ikiye ayrılmıştır. Önce tüm çekirdeği bitir; vaktin kalırsa opsiyonellere geç. **Az ama sağlam** her zaman çok-ama-yarım'dan iyidir.

No	Gereksinim	Öncelik
F1	Ajan YAML config okur ve CPU/RAM/Disk check'lerini periyodik koşar	Zorunlu
F2	Eşik aşımında seviyeli (INFO/WARN/ERROR) olay üretir	Zorunlu
F3	Ajan olayları toplayıcıya push eder; erişilemezse yerelde kuyruklar ve retry yapar	Zorunlu
F4	Gönderilen veri imzalı/hash'li; toplayıcı bütünlüğü doğrular	Zorunlu
F5	Toplayıcı olayları saklar ve REST API ile sunar	Zorunlu
F6	Dashboard: filo görünümü + host detayı + alarm geçmişi	Zorunlu
F7	<code>pulsectl</code> : status & config validate	Zorunlu
F8	Rapor indirme (CSV zorunlu, PDF opsiyonel)	Zorunlu / Ops.
F9	Eşik aşımında e-posta veya webhook bildirimi	Opsiyonel
F10	Genişletilebilir check tipi (özel komut / HTTP endpoint)	Opsiyonel
F11	Docker Compose ile tek komutla tüm sistemi ayağa kaldırma	Opsiyonel
F12	Dashboard'dan eşik düzenleme (yalnız görüntüleme değil)	Opsiyonel
F13	Gerçek asimetrik imza (ör. ed25519 / PGP) ile batch imzalama	Opsiyonel

7 · Teknik Gereksinimler & Kısıtlar

Konu	Beklenti
Diller	Ajan, Toplayıcı, CLI → Go . Dashboard → React + TypeScript .
Veri deposu	SQLite (kurulumu basit). PostgreSQL opsiyonel ileri hedef.
İletişim	Ajan → Toplayıcı ve Dashboard → Toplayıcı arası HTTP/REST + JSON .
Sürüm kontrol	Git . Anlamlı commit mesajları, düzenli commit'ler. Repo bir README ile başlar.
Kod kalitesi	Anlaşılır isimler, küçük fonksiyonlar, temel hata yönetimi, kritik akışlar için birkaç birim test .
Yapılandırma	Sırlar (token vb.) koda gömülmez; config veya ortam değişkeninden okunur.
Loglama	Ajan ve toplayıcı ne yaptığını anlaşılır biçimde loglar (seviyeli).
Kurulabilirlik	Başkası repo'yu klonlayıp README 'deki adımlarla sistemi ayağa kaldırabilmeli .

▢ ARAŞTIRILACAK SORULAR

- Go modülleri (**go.mod**) nasıl çalışır? Proje nasıl derlenir, tek binary üretmenin avantajı ne?
- Birim test (unit test) nedir, neyi test etmeli neyi etmemeli? Go'da **testing** paketi nasıl kullanılır?
- Sırları (secret) kodda tutmak neden tehlikeli? Alternatifler neler (env var, config, vault)?

8 · Yol Haritası (Aşamalar)

Gün gün değil, **aşama aşama** ilerle. Her aşamanın sonunda çalışan ve gösterilebilir bir şey olmalı — “yatay dilimler” halinde büyüt, tek seferde her şeyi bitirmeye çalışma.

Aşama 0 — Keşif & Tasarım (başlangıç)

Araştırma sorularının önemli kısmını yanıtla. Mimariyi bir kağıda/çizime dök: hangi parça hangi veriyi nasıl gönderiyor? Olay JSON şemasını taslak olarak tasarla. Teknoloji seçimlerini **gerekçeleriyle** yaz. Kısa bir tasarım notu çıkar.

Aşama 1 — Ajanın çekirdeği (erken)

YAML config okuma + CPU/RAM/Disk check'leri + eşik/olay üretimi. Henüz gönderim yok; olayları ekrana/log'a bas. İlk “çalışan şey”.

Aşama 2 — Toplayıcı + API + kalıcılık (orta)

Toplayıcı olay alır, SQLite'a yazar, sorgu uç noktalarını sunar. Ajani toplayıcıya bağla (push). İmza/bütünlük doğrulamayı ekle. Dayanıklılık: retry + yerel kuyruk.

Aşama 3 — Dashboard (orta-geç)

React+TS ile filo görünümü + host detayı + alarm geçmişi. API'ye bağlan, gerçek veriyi göster. Temel grafik.

Aşama 4 — CLI + Rapor + Cila (geç)

`pulsectl status` / `config validate`. CSV rapor. Hata yönetimi, loglama, README, birkaç birim test. Kurulumu baştan sona dene.

Aşama 5 — İleri hedefler & Sunum (kalan süre)

Vakit kaldıysa opsiyonel gereksinimler (Docker Compose, e-posta/webhook, gerçek imza). Demo senaryosunu hazırla, mimari dokümanı tamamla, sunuma çalış.

Altın kural

Her aşamanın sonunda kod çalışır ve gösterilebilir durumda olsun. “Bitmemiş 5 parça” yerine “biten 3 parça” hedefle. Takıldığında en geç yarım günde mentorundan yardım iste — takılıp kalmak değil, *doğru soruyu sormak* makbuldür.

9 · Teslimatlar & “Bitti” Tanımı

Staj sonunda teslim edilecekler

1. **Git repo:** dört bileşenin kaynak kodu + anlaşılır [README](#) (nasıl kurulur/çalıştırılır).
2. **Çalışan demo:** en az bir ajan → toplayıcı → dashboard akışını canlı gösterebilmek.
3. **Mimari dokümanı (2-4 sayfa):** parçalar, veri akışı, önemli kararlar ve *neden*’leri, araştırma sorularına verdiği cevapların özeti.
4. **Kısa sunum:** ne yaptın, neyi neden seçtin, nerede zorlandın, daha çok vaktin olsa ne eklerdin.

Bir özellik ne zaman “bitmiş” sayılır?

- Mutlu senaryoda çalışıyor **ve** beklenen hata durumlarında makul davranıyor (çökmüyor, anlamlı mesaj veriyor).
- Kod okunur; sırlar gömülü değil; ilgili akış loglanıyor.
- README’deki adımlarla başkası da çalıştırabiliyor.

10 · Değerlendirme Kriterleri (mentor için)

Not “ne kadar çok özellik yaptın” değil; **mühendislik olgunluğu** üzerinedir. Yarım kalan bir opsiyonel özellik, temiz düşünülmüş bir çekirdekten daha az değerlidir.

Kriter	Neye bakılır	Ağırlık
Çalışan çekirdek	Zorunlu gereksinimler (F1-F8) uçtan uca çalışıyor mu?	30%
Mimari & tasarım kararları	Parçaları doğru mu ayırmış; kararlarını gerekçelendirebiliyor mu?	25%
Araştırma & öğrenme	Sorulara kendi araştırmasıyla anlamlı cevaplar üretmiş mi?	15%
Kod kalitesi	Okunabilirlik, hata yönetimi, temel testler, git hijyeni.	15%
Dokümantasyon & iletişim	README, mimari doküman, sunum netliği; ilerleme paylaşımı.	10%
İnisiyatif & ileri hedefler	Opsiyonelleri denemesi, kendi fikirlerini katması.	5%

11 · Başlangıç Kaynakları

Bunlar başlangıç noktası; ezberlemek değil, ihtiyaç duydukça *bakmayı öğrenmek* için. En değerli beceri: doğru soruyu arayıp güvenilir kaynaktan cevaplamak.

- **Go:** “A Tour of Go”, Effective Go, standart kütüphane (`net/http` , `encoding/json` , `time` , `testing`).
- **Sistem metrikleri (Go):** `gopsutil` kütüphanesi — CPU/RAM/disk okuma.
- **REST API tasarımı:** HTTP metodları ve durum kodları; iyi API tasarım rehberleri.
- **React + TypeScript:** resmi React dokümanı (yeni), TypeScript handbook, bir grafik kütüphanesi (Recharts/Chart.js).
- **Kavramsal:** data diode / one-way transfer nedir; push vs pull monitoring; message queue & retry temelleri; veri bütünlüğü (hash, dijital imza).
- **Araçlar:** Git temelleri, SQLite, (opsiyonel) Docker & Docker Compose.

Son söz

Bu proje kasıtlı olarak “her şeyi çözülmüş” verilmedi. Boşluklar senin araştırıp karar vereceğin yerler — asıl öğrenme orada. Kararsız kaldığında bir yol seç, **neden seçtiğini yaz**, ve ilerle. Başarılar! □